

VOLUME 3 OF 3 · EXECUTION

SADHANA

Build playbook: module-wise sprints with ready-to-use Claude prompts

14 weeks · 5 sprints, 8 modules, 62 tasks

Every task · Deliverable, prompt, verify, done-when

Prompt library · Build, debug, review, test, plan

రేపు ఉదయం ఏమి చేయాలో

ఇక్కడ ఉంది — T01 నుంచి మొదలు

Contents

What this document is	3
PART A — How to work with Claude on this project	3
A.1 The context primer	4
A.2 Session hygiene	4
A.3 The four prompt patterns	5
A.4 Anti-patterns — do not do these	6
PART B — Module map	7
SPRINT 0 — Foundation	8
Sprint 0 exit gate	15
SPRINT 1 — Exam Hub (M1)	16
Sprint 1 exit gate	19
SPRINT 2 — Notifications and Ingestion (M2, M3)	21
Sprint 2 exit gate	26
SPRINT 3 — Quiz, Streaks, Push (M4, M5)	27
Sprint 3 exit gate	32
SPRINT 4 — Polish, Compliance, Launch (M6, M7)	33
Sprint 4 exit gate	36
PART C — Reusable prompt library	37
PART D — Weekly ritual	39
PART E — The five decisions you will be tempted to reverse	40

Field	Detail
Document	Volume 3 of 3 — Execution Playbook
Companions	Vol 1 — R&D and Business Case · Vol 2 — Implementation Documentation
Version	1.0
Date	September 2026
Owner	Lokesh
Covers	Sprint 0 → Launch, 14 weeks, 8 modules, 62 tasks

What this document is

Volume 1 said why. Volume 2 said how. **This one says what to do on Monday morning.**

Every task below has four parts:

Part	What it gives you
Deliverable	The exact artifact that must exist when the task is done
Claude prompt	Copy-paste text to hand to Claude. Written to produce working code, not a tutorial.
Verify	The command or check that proves it works
Done when	The condition to tick the task off. No ambiguity.

Work top to bottom. Do not skip ahead — later tasks assume earlier ones exist.

PART A — How to work with Claude on this project

సారాంశం: ప్రతి సెషన్‌లో సందర్భం ఇవ్వాలి. లేకపోతే Claude ప్రతిసారీ కొత్తగా ఊహిస్తుంది.

A.1 The context primer

Claude does not remember your project between sessions unless you tell it. **Paste this block at the start of every new chat.** It costs you 20 seconds and saves you an hour of wrong-shaped code.

PROJECT CONTEXT – SADHANA

Product: Telugu-first portal for Indian government/private job aspirants.
Modules: notification feed with eligibility matching, exam hub pages (SEO), daily quiz with streaks, material library, doubt community, test series.

Stack:

- Laravel 12, PHP 8.3
- Livewire 3 + Alpine.js (server-rendered – SEO is the business model, so no SPA, no client-side routing)
- Filament 3 for admin
- MySQL 8, Redis (cache/queue/streaks), Meilisearch (per-locales indexes)
- Cloudflare R2 for files, Cloudflare CDN
- Tailwind CSS
- Pest for tests

Hard rules:

1. Multi-language (te/en) is core, not an afterthought. Translatable content uses spatie/laravel-translatable JSON columns. Slugs are NEVER translated.
2. Every cache key must include app()->getLocale().
3. Target device is a Rs 8,000 Android on 3G. JS budget under 100 KB. Telugu text runs 15-30% longer than English – layouts must survive that.
4. Modular monolith. Modules talk via service classes, never by reaching into each other's models.
5. No pirated content anywhere in the system.

Code style: Laravel conventions, strict types, service classes for logic, form requests for validation, Pest for tests. Prefer readable over clever.

Today I am working on: [FILL IN]

A.2 Session hygiene

Rule	Why
One task per session	A chat that has built four things gives worse answers on the fifth. Start fresh per task.
Paste real code, not descriptions	"My model has some fields" produces guesses. Paste the migration.
Paste real errors, full stack trace	The line number is usually the whole answer.
Say what you already tried	Otherwise you get the same suggestion you already ruled out.
Ask for the file path	"Give me the full file including the path it goes in" — saves you deciding.
Reject the first answer if it is generic	Say "this is generic, use my actual schema above". It will.

A.3 The four prompt patterns

Almost every prompt you will write is one of these four shapes.

Pattern 1 — BUILD

```
Build [component name].

CONTEXT: [paste the relevant migration / model / existing code]

REQUIREMENTS:
- [requirement 1]
- [requirement 2]

CONSTRAINTS:
- [what it must not do]

OUTPUT: complete file(s) with the path each one goes in.
Include the Pest test.
```

Pattern 2 — DEBUG

```
This is broken.

WHAT I EXPECTED: [x]
WHAT HAPPENED: [y]

ERROR:
[full stack trace]

CODE:
[the actual file]

ALREADY TRIED: [a, b]

Find the cause. Do not rewrite the whole file – show me the fix.
```

Pattern 3 — REVIEW

```
Review this before I merge.

[code]

Check specifically for: N+1 queries, missing locale in cache keys,
missing authorization, unvalidated input, anything that breaks
on a slow connection.

Rank findings by severity. Ignore style nitpicks.
```

Pattern 4 — EXPLAIN / DECIDE

```
I need to decide between [A] and [B] for [purpose] in this project.

My constraints: [scale, team size, deadline]
```

Give me one committed recommendation and the two strongest arguments against it. No "it depends".

A.4 Anti-patterns — do not do these

Anti-pattern	What happens
"Build me the whole notification module"	You get 900 lines you cannot review, half of which do not fit your schema
"Fix my code" with no error	Guesswork
Accepting code you do not understand	You cannot debug it at 2 AM on notification day
Asking for options when you need a decision	You spend the session comparing instead of building
Building Sprint 3 features during Sprint 1	Scope creep is how this project dies

PART B — Module map

సారాంశం: ఎనిమిది మాడ్యూల్స్. ఏ మాడ్యూల్ ఏ స్ప్రింట్‌లో వస్తుందో ఇక్కడ ఉంది.

#	Module	Sprint	Tasks	Why it exists
M0	Foundation — infra, i18n, auth, design system	S0	T01-T12	Everything else sits on it
M1	Exam Hub	S1	T13-T22	SEO engine — 60% of acquisition
M2	Notification Feed + Eligibility	S2	T23-T35	The core value proposition
M3	Ingestion / Scrapers	S2	T36-T39	Content velocity
M4	Daily Quiz + Streaks	S3	T40-T50	Retention engine
M5	Push + WhatsApp	S3	T51-T54	Distribution
M6	PWA + Performance	S4	T55-T57	Reach on cheap phones
M7	Compliance + Launch	S4	T58-T62	Legal survival

Deferred to Phase 2 — do not build these now: material library, doubt community, study circles, test series, premium subscription. They are documented in Vol 2 and they wait until D7 retention is proven above 20%.

SPRINT 0 — Foundation

సారాంశం: పునాది. ఇక్కడ తొందరపడితే మిగతా 12 వారాలు బాధపడాలి.

Weeks 1–2 · Module M0 · 12 tasks Sprint goal: an admin can create a bilingual exam in Filament and it renders correctly in Telugu on a real phone.

T01 — Competitor teardown

Deliverable: a document of screenshots and friction notes from Adda247, Testbook, Sakshi Education.

No code this task. Do it on a real budget Android, not an emulator.

Steps: 1. Install all three apps. Complete full signup on each. 2. Switch each to Telugu. Screenshot every place it silently falls back to English. 3. Follow one live notification across all three plus two Telegram channels. Time each. 4. Write down every moment you were confused or annoyed.

Claude prompt (after you have the notes):

I did a competitor teardown of three Indian exam-prep apps. Here are my raw notes:

[paste your notes]

Turn this into:

1. A prioritised list of UX failures I should NOT repeat
2. Three things they do well that I should copy
3. The single biggest gap I can exploit

Be specific and concrete. No generic UX advice.

Verify: you have at least 30 screenshots and 20 friction notes. **Done when:** the friction list is written down and pinned where you will see it.

T02 — Project scaffold

Deliverable: a running Laravel 12 app with Livewire, Filament, Tailwind, Pest.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: initial project scaffold.

Give me the exact command sequence to set up a fresh Laravel 12 project with:

- Livewire 3
- Filament 3 (admin panel on /admin, separate guard)
- Tailwind CSS with Vite
- Pest 3 replacing PHPUnit
- Laravel Pint and PHPStan level 6
- spatie/laravel-translatable, laravel/scout, meilisearch-php, spatie/laravel-sitemap, intervention/image, sentry/sentry-laravel

Then give me:

1. The final composer.json
2. phpstan.neon config
3. pint.json config
4. .env.example with every variable this project needs

Commands only, no explanation of what Laravel is.

Verify: `php artisan serve` loads, `/admin` shows a Filament login, `php artisan test` passes. **Done when:** committed to `main` and CI runs green.

T03 — Database migrations

Deliverable: every core table from Vol 2 Chapter 4, migrated.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: database migrations.

Here is my target schema:

[paste the SQL from Vol 2 Chapter 4 – do this in 3 batches:

- batch 1: users, profiles, user_exam_preferences, exam_categories, exams, exam_cutoffs
- batch 2: notifications, notification_saves, questions, daily_quizzes, quiz_attempts, streaks
- batch 3: materials, posts, answers, votes, content_translations, ad_campaigns, ad_events, scrape_sources, translation_queue, push_tokens]

Convert these to Laravel 12 migration files. Rules:

- One migration per table, correctly ordered for foreign keys
- Use `$table->json()` for translatable columns
- Keep every index I specified – they are deliberate
- Add `softDeletes` where I marked `deleted_at`
- Use `foreignId()->constrained()` where it reads cleanly

Give me the full file contents and the filename for each.

Run it in three sessions, one batch each. A single session producing 22 migrations will drift.

Verify: `php artisan migrate:fresh` runs clean. Check indexes with `SHOW INDEX FROM notifications;`

Done when: `migrate:fresh` and `migrate:rollback` both work without error.

T04 — Models, relationships, factories

Deliverable: Eloquent models with relationships, casts, and factories.

Claude prompt:

```
[PASTE CONTEXT PRIMER]

Today I am working on: Eloquent models.

Here are my migrations:
[paste 4-5 related migrations]

Generate the matching models with:
- HasTranslations trait + $translatable array on translatable columns
- All relationships (belongsTo, hasMany, belongsToMany) both directions
- $casts for dates, json, booleans
- SoftDeletes where present
- Query scopes I will obviously need: scopePublished, scopeActive,
  scopeForLocale
- A factory for each with realistic Indian exam data (real exam names,
  real Telugu strings, realistic vacancy counts and dates)

Full file contents with paths.
```

Verify:

```
php artisan tinker
>>> Exam::factory()->count(5)->create()
>>> Exam::first()->notifications
```

Done when: every relationship resolves and factories produce data that looks real.

T05 — Locale config and routing

Deliverable: `/te/...` and `/en/...` routing working.

Claude prompt:

```
[PASTE CONTEXT PRIMER]

Today I am working on: multi-language routing.

Build:
1. config/locales.php — supported locales with name, native name,
  active flag. Telugu and English active; Hindi and Tamil defined
  but inactive.
2. A SetLocale middleware that:
  - reads the {locale} route parameter
  - validates against active locales, falls back to default
  - App::setLocale()
  - URL::defaults(['locale' => $locale]) so route() never needs
    the locale passed manually
  - persists to a 1-year cookie
  - syncs the logged-in user's preferred_locale without firing events
3. routes/web.php with a locale-prefixed group, where clause on the
```

```
parameter, and named routes for: home, exams.index, exams.show,
notifications.index, notifications.show, quiz.today
4. Root "/" resolution: cookie first, then Accept-Language, then 'te'
5. A language switcher Blade component that links to the SAME page in
the other locale, not the homepage

Full files with paths, plus Pest tests for the middleware.
```

Verify: `/te/exams` and `/en/exams` both load; switching language on an exam detail page stays on that exam. **Done when:** the switcher preserves the current route and its parameters.

T06 — UI translation files

Deliverable: `lang/te.json` and `lang/en.json`.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: UI string translations.

Generate `lang/te.json` for a government-job portal. Use ENGLISH SENTENCES as keys (not dot notation) so missing keys fall back readably.

Cover these areas – roughly 120 strings:
navigation, auth/OTP, profile form, notification feed, eligibility badges, exam hub sections, quiz UI, streaks, errors, empty states, buttons, dates.

Critical translation rules:

- "Notification" stays "నోటిఫికేషన్", NOT "ప్రకటన" – that is what aspirants actually search for
- "Group 2" stays "గ్రూప్ 2"
- Keep exam names and abbreviations (SSC, RRB, TGPSC) untranslated
- Use the everyday register aspirants use, not formal literary Telugu
- Support Laravel `:placeholder` syntax where a count or name is inserted

Output the full JSON. Also give me `lang/en.json` with identical keys mapping to themselves.

Verify: set locale to `te`, confirm no raw keys appear in the UI. **Done when:** both files exist and a Telugu reader confirms the register sounds natural, not machine-translated.

T07 — Design system and Telugu typography

Deliverable: Tailwind config, base components, Telugu-safe type scale.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: design system.

Build a Tailwind + Blade design system for a Telugu exam portal.

Constraints:

- Primary device: Rs 8,000 Android, 360px viewport, 3G
- Telugu strings run 15-30% longer than English and glyphs are taller – every component must survive that without clipping
- Self-hosted subset Noto Sans Telugu; NOT Google Fonts CDN
- Numerals stay Latin (2026, Rs 28,940) – Telugu numerals are not read fluently by this audience

Give me:

1. `tailwind.config.js` – colour scale (trustworthy, not playful; this is a career decision tool), type scale, spacing
2. The `@font-face` block plus `html[lang="te"]` rules with line-height 1.85 for Telugu and 1.6 for English
3. Blade components: `x-button` (3 variants), `x-card`, `x-badge` (green/amber/grey for eligibility), `x-input`, `x-select`, `x-empty-state`, `x-skeleton`
4. A base layout with the header, bottom nav (mobile-first), and language switcher

Every component must be shown rendered with BOTH English and Telugu text so I can see the layout hold.

Verify: open every component with Telugu content at 360px. Nothing clips or overflows. **Done when:** a Telugu string 30% longer than its English equivalent does not break any component.

T08 — Phone OTP authentication

Deliverable: working phone-based signup and login.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: phone OTP auth.

Build phone-number authentication for Indian users:

1. Livewire component: enter phone -> receive OTP -> verify -> logged in
2. Indian phone validation (propagandistas/laravel-phone, IN region)
3. 6-digit OTP, 10-minute expiry, stored hashed in cache not the database
4. Rate limits: 3 OTP requests per phone per hour, 5 verify attempts per OTP
5. An `OtpChannel` interface with two drivers: `LogDriver` (development) and `WhatsAppDriver` (production, Meta Cloud API) – I will wire WhatsApp later
6. New user -> create user + empty profile -> redirect to profile setup
Returning user -> redirect to intended
7. All copy through `__()` so it works in Telugu

No passwords anywhere. Full files plus Pest tests covering rate limits and expiry.

Verify: signup with a real number, check the OTP in `storage/logs/laravel.log`, confirm rate limits by spamming. **Done when:** rate limiting actually blocks, and OTPs expire.

T09 — Profile setup (drives eligibility)

Deliverable: the profile form whose fields feed the eligibility engine.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: user profile setup.

Build a multi-step profile form. These fields exist ONLY because the eligibility engine needs them:

date_of_birth, gender, category (general/obc/sc/st/ews), is_pwd, is_ex_serviceman, highest_qualification, qualification_stream, state, district

Requirements:

- 3 steps: basic -> qualification -> exam interests
- Progress indicator; each step saves so nothing is lost on a dropped connection
- District dropdown populated from the selected state (TS and AP districts)
- Exam interests: multi-select from the exams table, searchable
- Skippable, but show a persistent banner explaining that eligibility matching needs it
- DPDP consent: itemised checkboxes with purpose stated, store consent version and timestamp
- Mobile-first, works at 360px, all copy through __()

Full files, Form Request validation, Pest tests.

Verify: complete the flow on a phone; drop the connection mid-way and confirm the earlier step survived. **Done when:** a completed profile has every field the `EligibilityService` reads.

T10 — Filament admin resources

Deliverable: the content team can manage exams and notifications.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: Filament admin.

Build Filament 3 resources for Exam and ExamNotification.

Key requirement – translatable fields need a TAB per active locale, generated from `config('locales.supported')`, not hardcoded. English required, Telugu optional but visibly flagged when missing.

ExamResource: name, slug (auto from English name, never translated), category, conducting body, official website, description, exam pattern, syllabus, meta title/description, FAQ repeater.

ExamNotificationResource: all fields from my notifications table. Group into sections: Basic, Vacancy, Eligibility, Dates, Fees, Links, Status. The Eligibility section needs a key-value field for age_relaxation by category.

Both need:

- A status column with colour coding
- A "missing Telugu translation" filter – content ops need this daily
- Bulk publish action
- A required confirmation modal on publish that says
"Have you verified dates and eligibility against the official PDF?"

Full resource files.

Verify: create a bilingual exam end to end in the panel. **Done when:** the publish confirmation cannot be bypassed.

T11 — CI/CD pipeline

Deliverable: push to `main` deploys automatically.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: CI/CD.

Build two GitHub Actions workflows for a Laravel 12 app deploying to a Hostinger VPS over SSH.

ci.yml on every push and PR:

PHP 8.3, MySQL 8 and Redis services, composer install, migrate, pint --test, phpstan, pest --parallel, npm build, and a step that FAILS the build if the gzipped JS bundle exceeds 100 KB.

deploy.yml on push to main:

maintenance mode, pull, composer install --no-dev, migrate --force, npm build, config/route/view cache, queue restart, supervisor restart, up. Must roll back and exit non-zero if migration fails.

Also give me the supervisor config for queue workers with priority queues: high, push, default, low.

Verify: push a trivial change and watch it deploy. **Done when:** a failing test blocks deployment.

T12 — Monitoring baseline

Deliverable: you find out about breakage before your users do.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: monitoring.

Set up for a Laravel 12 production app:

1. Sentry with release tracking and user context (id only, never phone or date of birth – DPDP)

2. Laravel Pulse
3. A HealthCheckController at /up returning JSON with database, redis, meilisearch, queue depth, and last successful scrape time
4. An artisan command sadhana:health that alerts when: error rate > 1% for 5 min, p95 response > 2s, queue depth > 10000, any scraper failed 3 consecutive runs
5. Scheduler entries for all of it

Full files.

Verify: throw a deliberate exception, confirm it lands in Sentry. **Done when:** /up reports honestly when you stop Redis.

Sprint 0 exit gate

- ☐ `migrate:fresh --seed` produces a working bilingual dataset
- ☐ `/te/` and `/en/` both render
- ☐ An admin creates an exam in Filament and it appears on the site in Telugu
- ☐ Signup by phone works end to end
- ☐ Push to main deploys automatically
- ☐ A page renders correctly on a real budget Android

Do not start Sprint 1 until every box is ticked.

SPRINT 1 — Exam Hub (M1)

సారాంశం: ఈ పేజీలో గూగుల్ నుంచి ఉచిత ట్రాఫిక్ తెస్తాయి. అందుకే ఇవి మొదట.

Weeks 3–4 · 10 tasks Sprint goal: 10 exam pages live, indexable, ranking-ready in both languages.

T13 — Exam listing page

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: exam listing page.

Build `/[locale]/exams`:

- Grouped by `exam_categories` with category icons
- Each card: name, conducting body, live-notification count badge, next important date
- Search box filtering client-side over already-loaded exams (no round trip – assume 3G)
- Filters: state (TS/AP/National), qualification level
- Server-rendered for SEO; Livewire only for the filters
- Cached 6 hours with locale in the cache key

Controller, Livewire component, Blade view, and a Pest feature test asserting a Telugu exam name renders on the `/te/` route.

Verify: view page source — exam names must be in the HTML, not injected by JS. **Done when:** disabling JavaScript still shows the full list.

T14 — Exam detail page

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: exam detail page – this is my main SEO asset.

Build `/[locale]/exams/{slug}` with these sections IN THIS EXACT ORDER:

1. H1: exam name + current year
2. Latest notification status banner
3. Quick facts table (body, vacancies, dates, qualification)
4. Eligibility
5. Exam pattern table
6. Syllabus – expandable per subject, with PDF download
7. Previous year papers
8. Cutoff trends table by year and category
9. Recommended free material
10. Top community questions for this exam (leave a placeholder – Phase 2)

- 11. Related exams
- 12. FAQ accordion

Requirements:

- Fully server-rendered. Alpine only for accordions.
- Eager load everything – I want ONE page, not an N+1 disaster
- Cache 6 hours, locale in the key
- View counter incremented after the response, never blocking it
- Semantic HTML: article, section, proper heading hierarchy

Controller, Blade views (one partial per section), Pest test.

Verify: Laravel Debugbar — query count must be under 10. **Done when:** query count is under 10 and page loads under 500ms uncached.

T15 — SEO infrastructure

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: SEO. This is my primary acquisition channel – get it exactly right.

Build:

1. App\Support\SeoBuilder with static methods forExam(), forNotification(), forHome(). Each returns title, description, canonical, and an alternates array keyed by locale.
2. A seo Blade partial rendering title, meta description, canonical, hreflang for every active locale plus x-default pointing to English, and Open Graph tags.
3. Structured data builders:
 - Course for exams
 - JobPosting for notifications (this gets me into Google Jobs – include validThrough, hiringOrganization, jobLocation, baseSalary, totalJobOpenings)
 - FAQPage for exam FAQ blocks
 - BreadcrumbList site-wide
4. Sitemap generation: separate sitemap per locale per content type, under a sitemap index. Nightly artisan command.
5. robots.txt allowing everything except /admin and /api.

Full files. Show me the rendered HTML head for one exam page in Telugu so I can eyeball it.

Verify: paste a rendered page into Google's Rich Results Test. **Done when:** JobPosting and FAQPage both validate with zero errors.

T16 — Cache and purge layer

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: caching.

Build:

1. App\Support\Locale::cacheKey(\$key) that appends the current locale. Every cache call in this project goes through it.
2. Model observers on Exam and ExamNotification that, on save or delete, forget the local cache for EVERY active locale and dispatch a Cloudflare purge job for the corresponding URLs.
3. A PurgeCloudflareUrl queued job using the Cloudflare API, with retry and failure logging.
4. Cloudflare page rules I should configure (tell me the exact settings): exam pages 6h, notification detail 1h, anonymous feed 5min, never cache anything under /admin or for authenticated sessions.

Full files plus a Pest test proving a Telugu cache entry is cleared when the model is saved.

Verify: edit an exam, hard-refresh both `/te/` and `/en/` — the change must appear on both. **Done when:** no stale locale is possible after an edit.

T17-T20 — Content population

Not code. This is the sprint's real work.

Populate 10 exams completely, in both languages: TGPSC Group 2, TGPSC Group 4, APPSC Group 2, SSC CGL, SSC CHSL, RRB NTPC, RRB Group D, TS Police Constable, AP Police Constable, DSC/TET.

Per exam: description, eligibility, pattern, full syllabus, 5 years of cutoffs, 3 previous papers, 8 FAQs — English then Telugu.

Claude prompt (per exam, use it as a drafting assistant, not an oracle):

[PASTE CONTEXT PRIMER]

Today I am working on: content for [EXAM NAME].

I am pasting the official notification PDF text below. Extract and structure:

1. A 2-sentence description
2. Eligibility (qualification, age with category-wise relaxation)
3. Exam pattern as a table (papers, marks, duration, negative marking)
4. Full syllabus by subject and topic
5. 8 FAQs aspirants actually ask, with answers

Rules:

- Use ONLY what is in the text below. If something is not stated, write "Not specified in notification" — do NOT fill gaps from memory.
- Flag anything ambiguous for my manual check.
- Give me English first. Then Telugu, keeping exam names, "Group 2", and abbreviations untranslated.

[PASTE PDF TEXT]

Verify: cross-check every date and eligibility line against the official PDF yourself. Twice. **Done when:** 10 exams complete in both languages, each with a "Last verified" date and a link to the official source.

T21 — Search integration

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: Meilisearch.

Wire Laravel Scout to Meilisearch with SEPARATE INDEXES PER LOCALE (notifications_te, notifications_en, exams_te, exams_en) – Telugu tokenisation differs from English and a shared index wrecks relevance for both.

Give me:

1. Searchable trait config on Exam and ExamNotification with searchableAs() returning the locale-suffixed index name and toSearchableArray() pulling the right translation
2. An artisan command that reindexes ALL locales, not just the current one
3. Index settings per locale: searchableAttributes, filterableAttributes, sortableAttributes, ranking rules with recency weighting
4. A global search Livewire component: debounced, grouped results by type, keyboard navigable, works at 360px

Full files.

Verify: search a Telugu term on `/te/`, an English term on `/en/`. Both must return sensible results. **Done when:** results differ appropriately by locale, proving separate indexes are live.

T22 — Search Console setup

No code. Verify the domain, submit each locale's sitemap separately, set international targeting, check the hreflang report for errors.

Done when: both sitemaps are submitted and hreflang reports zero errors.

Sprint 1 exit gate

- ☐ 10 exam pages complete in Telugu and English
- ☐ Rich Results Test passes on Course, JobPosting, FAQPage
- ☐ hreflang clean in Search Console
- ☐ Sitemaps submitted per locale

- ☐ Exam detail page under 10 queries, under 500ms uncached
 - ☐ Cache purges on edit across all locales
-

SPRINT 2 — Notifications and Ingestion (M2, M3)

సారాంశం: ఉత్పత్తి యొక్క గుండె. అర్హత ఇంజిన్ మన ప్రత్యేకత — దానికి టెస్టులు తప్పనిసరి.

Weeks 5–7 · 17 tasks Sprint goal: a user sets a profile and sees a personalised, eligibility-tagged feed.

T23 — Eligibility engine

This is the most important code in the product. Budget a full day. Test it hardest.

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the eligibility engine. This is the single highest-value piece of logic in my product – a wrong answer misleads someone about their career, so correctness matters more than elegance.

Build App\Services\EligibilityService with a check(ExamNotification, ?Profile) method returning:

```
['status' => 'eligible'|'partial'|'not_eligible'|'unknown',
 'matched' => [...], 'failed' => [...], 'score' => float]
```

Rules to implement:

1. QUALIFICATION – rank order: 10th=1, 12th=2, iti=2, diploma=3, degree=4, btech=4, mbbs=5, pg=5, phd=6. User rank >= required rank.
2. AGE – calculate against age_reference_date, falling back to apply_end_date, then today. Apply relaxation from the notification's age_relaxation JSON: category relaxation PLUS pwd PLUS ex_serviceman, cumulative.
3. DOMICILE – if allowed_states is set, user's state must be in it. Null means all-India.
4. GENDER – if gender_restriction is not 'any', must match.
5. Return 'partial' when some criteria pass – NEVER hide a notification entirely. Users may have relaxations I do not know about.
6. Return 'unknown' with no profile.

Then write EXHAUSTIVE Pest tests including these edge cases:

- SC candidate aged 33 for a max-age-30 post with sc:5 relaxation -> eligible
- PwD + OBC stacking both relaxations
- Birthday falling exactly on the reference date
- Notification with no age limits at all
- Notification with no qualification specified
- Profile with a null date_of_birth
- All-India notification vs a TS-only user
- Leap-year birth date

I want 95%+ coverage on this class. Show me the test file first, then the implementation.

Ask for tests first deliberately — it forces the edge cases into the open before the logic is written.

Verify: `php artisan test --filter=Eligibility --coverage` **Done when:** coverage above 95% and every listed edge case has a passing test.

T24 — Eligibility caching

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: eligibility caching.

Eligibility checks run for every notification in the feed on every load.

Make it cheap:

1. Cache the result per user+notification for 1 hour in Redis
2. Invalidate every entry for a user when their profile changes (observer on Profile)
3. Invalidate every entry for a notification when its eligibility fields change – but NOT when unrelated fields like view_count change
4. Batch method checkMany(Collection \$notifications, ?Profile) doing one Redis MGET instead of N round trips

Full files plus a test proving a profile update clears the cache.

Verify: load the feed twice, watch Redis hit count in Pulse. **Done when:** the second load makes zero eligibility computations.

T25 — Feed service

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the notification feed query.

Build `App\Services\FeedService::forUser(?User, array $filters, int $perPage)`.

Behaviour:

- Published only, not past `apply_end_date` (null end date = still open)
- User's preferred exams float to the top; everything else still visible below – never hide content
- Filters: `job_type`, `qualification`, `district` (match JSON column or null), `closing_soon` (next 7 days), `exam_id`
- Eager load exam
- Order: preferred exams first, then `published_at` desc
- Paginated

Then a companion method for the anonymous cached feed (5-minute cache, locale-keyed) that skips personalisation entirely.

Watch for: N+1 on exam, and the JSON district filter – show me the generated SQL for the district case.

Verify: Debugbar with 20 notifications loaded — query count must stay flat. **Done when:** query count does not grow with the number of notifications.

T26 — Feed UI

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the feed UI.

Build the notification feed Livewire component and Blade views.

Card contents: title (locale-aware), organisation, eligibility badge (green "You are eligible" / amber "3 of 4 match" / grey / none when logged out), vacancies, last date with a red highlight inside 3 days, Details and Save buttons.

Requirements:

- Filter bar: horizontally scrollable chips on mobile, not a dropdown
- "Only show what I am eligible for" toggle
- Infinite scroll via Livewire, with a skeleton loader
- Empty state that actually helps: "No notifications match. Try removing the district filter."
- Native ad slot every 5th card (component placeholder for now)
- Filter state in the query string so a filtered feed is shareable
- 360px first; Telugu text must not clip anywhere

Full files. Show me the card with a long Telugu title so I can check the layout holds.

Verify: open on a real phone with Telugu content and every badge state. **Done when:** no clipping, no horizontal scroll, badges legible at arm's length.

T27 — Notification detail page

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: notification detail page.

Build `/[locale]/notifications/{slug}`:

Sections: title + organisation, eligibility panel (each criterion listed with a tick or cross and the reason for each failure), vacancy breakdown table, full eligibility, important dates timeline with days-remaining, fee table by category, salary, how to apply, official PDF link, apply link, related notifications.

Mandatory:

- A visible "Last verified: [date]" with a link to the official source
- A disclaimer: always confirm against the official notification before applying
- A "Report an error" button that files to a moderation queue

- JobPosting structured data
- Sticky bottom bar on mobile with Apply and Save

Full files.

Verify: confirm the disclaimer and source link are visible without scrolling past the fold on mobile.

Done when: a user cannot miss the "verify officially" message.

T28 — Save and remind

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: save and deadline reminders.

Build:

1. Save/unsave toggle on the card and detail page, optimistic UI
2. /[locale]/saved page grouped by deadline proximity
3. On save, schedule reminders at 3 days and 1 day before apply_end_date
4. A SendDeadlineReminders scheduled job, idempotent – must never double-send
5. Skip reminders for notifications the user has already marked applied
6. Reminder copy in the user's preferred locale

Full files plus a test proving no duplicate sends when the job runs twice.

Verify: run the scheduler twice manually, confirm one notification. **Done when:** double execution produces no duplicates.

T29-T35 — Content and scraper work

T29-T31: Historical backfill. Load 12 months of past notifications through Filament. This is grind work and it is what makes the site look alive on day one.

T32 — TGPSC scraper

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the TGPSC notification scraper.

Build an abstract BaseScraper plus a TgpscScraper.

BaseScraper handles: fetch with a 30s timeout and an honest user agent (SadhanaBot/1.0 with a contact URL), skip URLs already in the database, create notifications with status = pending_review ALWAYS (never auto-publish – a wrong date costs a user their year), track consecutive_failures on the source, and alert ops after 3 consecutive failures because government sites change layout without warning.

TgpscScraper parses the notifications listing at tgpsc.gov.in and each

detail page. Here is the current HTML structure:

[paste the actual HTML you copied from the site]

Requirements:

- respect robots.txt and rate limit to 1 request per 5 seconds
- store facts (dates, vacancy counts, eligibility), not verbatim document copies
- always keep source_url so we can deep-link to the official page
- fail loudly, never silently

Full files plus a test using a saved HTML fixture.

T33 — APPSC scraper. Same prompt, different site and HTML.

T34 — AI extraction from PDFs

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: AI extraction from notification PDFs.

Build App\Services\NotificationExtractor that takes PDF text and returns structured data via the Anthropic API.

Fields: title, organisation, total_vacancies, min_qualification, min_age, max_age, age_relaxation (object by category), apply_start_date, apply_end_date, application_fee (object by category), salary_min, salary_max.

Non-negotiable rules to bake into the prompt:

- Return ONLY valid JSON, no markdown fences, no preamble
- null for anything not clearly stated
- NEVER infer or guess a date
- Extraction output ALWAYS lands in pending_review for human verification

Also handle: response parsing that survives stray markdown, a retry on malformed JSON, and a hard failure that logs rather than saving garbage.

Full file plus a test with a real notification's text as a fixture.

T35 — Review queue

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the notification review queue.

Build a Filament page for reviewing scraped notifications:

- Split view: extracted fields on the left, the source PDF in an iframe on the right
- Every field editable inline
- A checklist that must be ticked before publish:
 - dates verified, eligibility verified, fee verified, official link works
- Publish, Reject, and "Needs more info" actions
- Sorted by scrape time, oldest first

- A visible SLA warning when an item has been pending over 2 hours

Full files.

Verify: scrape a real notification, review it, publish it — under 5 minutes end to end. **Done when:** the checklist cannot be skipped and publish latency is measurable.

Sprint 2 exit gate

- ☐ Eligibility engine above 95% coverage, all edge cases green
 - ☐ Feed personalised and eligibility-tagged
 - ☐ 12 months of historical notifications loaded
 - ☐ Two scrapers running on schedule into the review queue
 - ☐ Publish latency under 60 minutes measured on a real notification
 - ☐ Save and deadline reminders working
-

SPRINT 3 — Quiz, Streaks, Push (M4, M5)

సారాంశం: రిటెన్షన్ ఇంజిన్. ఇది పని చేయకపోతే మిగతా అంతా వృధా.

Weeks 8-10 · 15 tasks Sprint goal: the 7 AM habit loop works end to end.

T40 — Question bank and admin

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the question bank.

Build a Filament resource for Question:

- Bilingual question text, options, and explanation – tabs per locale
- Correct answer as a radio over the ENGLISH options, index-linked so translations stay aligned
- Fields: exam, subject, topic, difficulty, is_current_affairs, source_year
- Bulk CSV import with a template download and per-row validation errors
- Filters: missing Telugu, never served, subject, difficulty
- A dashboard widget showing pool depth by category, because running out of questions breaks the daily quiz

Full files.

Verify: import 50 questions from CSV, including deliberately broken rows. **Done when:** bad rows are reported per-row without aborting the whole import.

T41 — Daily quiz generation

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: daily quiz generation.

Build the quiz:publish artisan command:

- Composition: 5 current affairs, 3 static GK/subject, 2 previous-year
- Exclude anything used in the last 60 days
- Only questions that HAVE a Telugu translation – a half-English quiz breaks the promise of the product
- Rotate the pool evenly: order by times_served ascending, then random
- If fewer than 10 qualify: do NOT publish a short quiz. Alert ops and exit non-zero.
- Idempotent – running twice for the same date is a no-op
- Increment times_served on the chosen questions
- Accept a --date option for backfilling

Scheduler at 06:45 IST. Full file plus tests for the pool-depletion path and the idempotency path.

Verify: run it twice for the same date; run it with an empty pool. **Done when:** neither case corrupts data and the empty case alerts.

T42 — Quiz UI

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the daily quiz UI.

Build the quiz-taking Livewire component:

- One question per screen, progress bar
- A PER-QUESTION language toggle (te/en) – a user may know the Telugu term but read the English option faster; do not force one language for the whole quiz
- Immediate feedback after answering, with the explanation
- Cannot change an answer once submitted
- Track time per question
- Resume where they left off if they close the app mid-quiz
- Works offline once loaded (cache the question set)
- Large tap targets – this is used on a phone, one-handed, often on a bus

Result screen: score, streak update, weak-area tag, leaderboard position, share button, "review answers".

Full files.

Verify: take the quiz on a phone, close the browser mid-way, reopen. **Done when:** progress is preserved and the language toggle works per question.

T43 — Streak service

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: streaks.

Build App\Services\StreakService::record(User).

Logic:

- Already active today -> no change, return current
- Last active yesterday -> increment
- Last active 2 days ago AND freezes_left > 0 -> consume a freeze and increment (users have exams, travel, emergencies – a streak that punishes real life gets abandoned and never restarts)
- Otherwise -> reset to 1
- Update longest_streak

- Reset freezes to 1 at the start of each month
- Write to a Redis sorted set for the weekly leaderboard
- Fire a StreakMilestoneReached event at 3, 7, 30, 100

All date logic in Asia/Kolkata, explicitly – not server default.

Exhaustive Pest tests: the freeze path, month rollover, a user acting at 23:59 then 00:01, timezone correctness, first-ever attempt.

Verify: travel the clock in tests across midnight and month boundaries. **Done when:** no timezone-dependent test failures.

T44 — Leaderboards

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: leaderboards.

Build weekly leaderboards on Redis sorted sets:

- Three scopes: state-wide, district, and the user's exam cohort
- Show the top 20 plus the user's own rank with two neighbours above and below – everyone should see themselves, not just the top 20
- Weekly reset, previous week archived to MySQL
- Efficient rank lookup via ZREVRANK, not by loading the whole set
- Livewire component with a scope switcher

Full files.

Verify: seed 10,000 fake users; the page must still load under 300ms. **Done when:** rank lookup does not degrade with set size.

T45 — Shareable result image

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: the shareable quiz result image.

This is my cheapest acquisition channel – a WhatsApp-status image that makes people ask "what app is that".

Build a service generating a 1080x1920 JPEG using Intervention Image:

- Score, streak count, date, sadhana.study wordmark
- Telugu text rendered with Noto Sans Telugu (bundle the TTF)
- Designed for WhatsApp status: bold, readable at thumbnail size, safe margins
- Generated on a queue, not in the request
- Stored on R2, URL returned to the client
- Web Share API where supported, download fallback

Full file. Describe the visual layout before you code it so I can approve the design.

Verify: generate one, actually post it to WhatsApp status, look at it on a phone. **Done when:** it is readable at thumbnail size and the brand is legible.

T46-T50 — Content sprint

500 questions in the bank, both languages. 60 days of quizzes pre-scheduled. This is the largest single content push in the project and it is what lets you sleep during launch week.

Claude prompt (batch drafting):

[PASTE CONTEXT PRIMER]

Today I am working on: quiz question drafting.

Generate 20 multiple-choice questions for [SUBJECT] at [DIFFICULTY] for Telugu-state government exam aspirants.

Format per question: question, 4 options, correct index, a 2-sentence explanation. English first, then Telugu.

Rules:

- Match the style of actual TGPSC/APPSC/SSC papers
- Only verifiable facts – no opinion, no trivia with disputed answers
- Distractors must be plausible, not obviously wrong
- Keep exam terms, "Group 2", and abbreviations untranslated in Telugu
- Flag any question where you are less than fully certain of the answer

Output as CSV matching my import template.

Verify: you personally check every answer key. Every one. **Done when:** 500 questions imported, all answer keys verified by a human.

T51 — Web push

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: web push.

Build FCM web push for a Laravel PWA:

1. Permission request – asked at the right moment (after the first completed quiz, never on first page load)
2. Token storage and deactivation on failure
3. A SendPush queued job with retry and dead-token cleanup
4. A PushBudget service enforcing a HARD CAP of 5 pushes per user per day – over-notification is how you get permanently muted, and a muted user is a lost user
5. Per-type user preferences: daily quiz, new notifications, deadlines,

```

    streak warnings
6. Locale-aware payloads
7. Service worker handler with deep links

Full files including the service worker code.

```

Verify: send yourself 6 pushes in a day; the 6th must be suppressed. **Done when:** the cap is enforced server-side, not client-side.

T52 — WhatsApp integration

Claude prompt:

```

[PASTE CONTEXT PRIMER]

Today I am working on: WhatsApp Cloud API.

I have Meta Cloud API experience from another project. Build for Sadhana:
1. A WhatsAppService: send template messages, send session messages,
   verify webhooks
2. Message templates to submit for Meta approval – draft the exact text
   in English and Telugu for: daily quiz link, new notification alert,
   deadline reminder, OTP
3. Webhook handling for delivery status and inbound messages
4. Opt-in and opt-out flows, with opt-out honoured immediately and
   permanently
5. Rate limiting to Meta's tier limits with a backoff

Full files plus the exact template submission text.

```

Verify: send yourself a template message; reply STOP; confirm opt-out sticks. **Done when:** opt-out is immediate and no further messages send.

T53 — Notification dispatch

Claude prompt:

```

[PASTE CONTEXT PRIMER]

Today I am working on: notification dispatch at scale.

Build SendNotificationAlert:
- Target users whose exam preferences match the notification
- chunkById(500), never load all users into memory
- Respect PushBudget, per-type preferences, and quiet hours (22:00-06:00)
- Locale-aware content per user
- Dispatch to the 'push' priority queue
- STAGGERED waves – 50,000 users at once will melt the app tier on
  notification day
- Idempotent: safe to re-run

Full file plus a test with 10,000 seeded users asserting memory stays flat.

```

Verify: run against 10,000 seeded users, watch memory in Pulse. **Done when:** memory does not grow with user count.

T54 — Streak reminders

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: streak reminder push at 21:00 IST.

Only for users who have an active streak and have NOT been active today.

Copy tone matters enormously here: my users are already under exam stress and family pressure. Encouraging, never guilt-inducing. No "you're about to lose everything". Draft 5 variants in English and Telugu and tell me which one you would ship and why.

Then build the scheduled job.

Verify: read the copy aloud. If it would make a stressed person feel worse, reject it. **Done when:** the copy is warm and the job respects the daily push budget.

Sprint 3 exit gate

- ☐ Daily quiz publishes automatically at 06:45
 - ☐ Quiz completable on a phone with per-question language toggle
 - ☐ Streaks correct across midnight, month, and timezone boundaries
 - ☐ Leaderboards fast at 10,000 users
 - ☐ Share image looks good on real WhatsApp status
 - ☐ 500 questions, 60 days scheduled
 - ☐ Push and WhatsApp both delivering, budget capped
-

SPRINT 4 — Polish, Compliance, Launch (M6, M7)

సారాంశం: చట్టపరమైన అంశాలు పూర్తి చేయకుండా లైవ్ కి వెళ్లకూడదు.

Weeks 11–14 · 8 tasks

T55 — PWA

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: PWA.

Build:

1. manifest.json — name, Telugu short_name, icons, standalone, theme colour, start_url with locale
2. A service worker:
 - app shell cached on install
 - stale-while-revalidate for exam pages
 - network-first with cache fallback for the feed
 - cache-first for fonts and static assets
 - an offline fallback page in the user's language
 - cached quiz question sets so a quiz survives a dropped connection
3. An install prompt shown after the third visit, never on the first
4. Update handling: notify, do not force-reload mid-quiz

Full files including the complete service worker.

Verify: install it, turn on airplane mode, open saved content. **Done when:** the offline experience is useful, not a browser error page.

T56 — Performance pass

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: performance.

Here are my Lighthouse results on a throttled 3G Moto G4 profile:

[paste the full report]

My budgets: LCP under 2.5s, TTI under 3.5s, JS under 100 KB gzipped, server p95 under 500ms.

Give me a prioritised fix list with the actual code changes. Rank by impact-per-hour-of-work. Ignore anything that will not move a real user on a real cheap phone.

Verify: re-run Lighthouse after each fix. **Done when:** every budget in Vol 2 Chapter 1.3 is met on a throttled profile.

T57 — Load test

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: load testing.

Write k6 scripts simulating notification-day traffic:

- Baseline: 500 concurrent
- Spike: 25,000 concurrent over 3 minutes (my traffic goes 20-50x when a major notification drops)
- Mixed journey: 60% feed browsing, 25% exam pages, 10% quiz, 5% search
- Logged-in and anonymous mix

Also give me the degradation strategy: when p95 exceeds 2s, what do I shed first, second, third? I would rather serve a degraded site than no site.

Scripts plus a runbook.

Verify: run against staging. **Done when:** the system survives 50x, and you know exactly what to shed and in what order.

T58 — DPDP compliance

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: DPDP Act 2023 compliance.

I store name, phone, date of birth, category, qualification, and district for lakhs of Indian users. Build:

1. Consent management: itemised checkboxes at signup in the user's language, with purpose stated per item; store consent version, IP, and timestamp
2. A working "Download my data" producing a JSON export of everything tied to the user
3. A working "Delete my account": hard-delete personal data within 30 days, retain only anonymised aggregates, with a confirmation flow and a grace period
4. A retention job anonymising accounts inactive for 3 years
5. An audit log on every admin read of user data

6. Privacy policy and terms content in English and Telugu, written plainly enough for a 22-year-old to actually understand

Full files. Flag anything where I need an actual lawyer rather than code.

Verify: create a test account, export, delete, confirm the data is gone from the database. **Done when:** export and deletion both work end to end and the audit log records admin access.

T59 — Legal pages

Publish privacy policy, terms, copyright policy, and takedown process — in both languages, with a real contact address and a stated 48-hour SLA.

Done when: all four are live and linked from the footer on every page.

T60 — Analytics

Claude prompt:

[PASTE CONTEXT PRIMER]

Today I am working on: analytics.

Set up event tracking for the metrics that actually matter (Vol 1 Chapter 10):

Events: signup_started, signup_completed, profile_completed, exam_preference_set, quiz_started, quiz_completed, streak_milestone, notification_viewed, notification_saved, eligibility_checked, search_performed, share_clicked, push_permission_granted

Requirements:

- No personally identifiable data in any event payload – DPDP
- Cohort retention tracking: D1, D7, D30
- A funnel view: organic visit -> signup -> profile -> first quiz
- A simple internal dashboard, not a third-party tool I have to pay for

Full files.

Verify: complete the full funnel yourself and confirm every event fires once. **Done when:** D1/D7/D30 retention is visible on the dashboard.

T61 — Beta

20 real users, recruited in person at coaching centres. Watch them use it. Do not explain anything — every time you need to explain, that is a bug.

Ship one fix per day based on what you saw.

Done when: a new user reaches their first completed quiz without you saying a word.

T62 — Launch

Work the checklist in Vol 2 Chapter 19. Launch on a day a major notification is expected — ride the spike rather than fighting for attention on a quiet Tuesday.

Done when: live, monitored, and someone you have never met has completed a quiz.

Sprint 4 exit gate

- ☐ PWA installable, offline mode useful
 - ☐ All performance budgets met on a throttled cheap-phone profile
 - ☐ Survived a 50x load test with a written degradation runbook
 - ☐ DPDP: consent, export, deletion all working
 - ☐ All four legal pages live in both languages
 - ☐ Analytics showing real D1/D7/D30
 - ☐ 20 beta users onboarded and their friction fixed
-

PART C — Reusable prompt library

సారాంశం: ఈ ప్రాంప్ట్లు ప్రతి వారం వాడతావు. బుక్మార్క్ చేసుకో.

Pre-merge review

Review this before I merge.

[code]

Check for, in this order:

1. N+1 queries
2. Cache keys missing app()->getLocale()
3. Missing authorization
4. Unvalidated user input
5. Anything that breaks on 3G or a 360px screen
6. Anything that breaks when the Telugu string is 30% longer

Rank by severity. Skip style nitpicks – Pint handles those.

Debugging

Broken.

EXPECTED: [x]

GOT: [y]

ERROR:

[full stack trace]

CODE:

[the file]

TRIED: [a, b]

Find the cause. Show me the fix, not a rewrite.

Writing tests for existing code

Write Pest tests for this:

[code]

Cover: happy path, every edge case you can find, failure modes, and specifically the Telugu/English locale variants.

I want to know what breaks, not to feel good about coverage numbers.

Performance investigation

This is slow: [endpoint], [n]ms p95.

CODE:

[controller, service, query]

DEBUGBAR:

[query log]

Rank the causes by impact. Give me the fix for the top one first.

Telugu content review

Review this Telugu translation for a government exam portal:

EN: [english]

TE: [telugu]

Check:

- Does it sound like everyday Telugu an aspirant uses, or like a machine?
- Are exam terms, "Group 2", and abbreviations left untranslated?
- Is the register right – respectful but not formal-literary?
- Any word here that people would NOT type into Google?

Give a corrected version if needed.

Weekly planning

[PASTE CONTEXT PRIMER]

Sprint [n], week [n]. Status:

DONE: [tasks]

IN PROGRESS: [task, % complete]

BLOCKED: [what and why]

Metrics: DAU [x], D7 [y]%, notification latency [z] min

Given my sprint exit criteria [paste them], tell me:

1. Am I on track – yes or no
2. What to cut if I am not
3. The one thing to do first tomorrow

Be blunt. I would rather cut scope than miss the gate.

PART D — Weekly ritual

సారాంశం: ప్రతి శుక్రవారం 30 నిమిషాలు. ఇది మానేస్తే స్కోప్ పెరిగిపోతుంది.

Every Friday, 30 minutes:

Step	Question
1	What shipped this week?
2	What slipped, and why specifically?
3	What are the numbers? DAU, D7, notification latency, error rate
4	Am I on track for the sprint exit gate?
5	If not — what gets cut , not what gets crunched
6	What is the single most important thing next week?

The rule that saves the project: when a sprint is at risk, cut scope. Never extend the sprint, never work the weekend. A sprint that slips two weeks becomes a project that slips two months.

PART E — The five decisions you will be tempted to reverse

Written down now, so future-you does not relitigate them at 1 AM.

#	Decision	The temptation	Why you hold
1	Community opens only after 5,000 DAU	"Let's launch with it, it looks better"	An empty forum signals a dead product and permanently damages first impressions
2	No pirated PDFs, ever	"Everyone does it, it would 10x our traffic"	One publisher notice ends the business. Traffic you cannot keep is not traffic.
3	Two languages at launch, not eight	"More languages, more users"	Thin pages destroy the SEO engine the entire model depends on
4	Human review on every date and eligibility	"The AI extraction is good enough now"	One wrong date costs a user a year and costs you their trust forever
5	Stop feature work if D7 is under 20% at Week 14	"Just one more feature will fix retention"	If the habit loop does not work, no feature built on top of it will

End of Volume 3.

Vol 1: why. Vol 2: how. Vol 3: what to do tomorrow morning. Start with T01.